

Lección 5 — Forensic Analysis

1. Introducción General de la Lección

El análisis forense digital permite descubrir, preservar, analizar e interpretar evidencia digital relacionada con incidentes de ciberseguridad, accesos no autorizados, malware, compromiso de sistemas, fugas de información y actividades sospechosas. Esta lección enseña cómo Python puede utilizarse para automatizar análisis forense, analizar tráfico de red, procesar logs, inspeccionar archivos PCAP, analizar metadata EXIF, realizar fingerprinting y trabajar con memory dumps y process dumps.

El enfoque de la lección es extremadamente práctico y orientado a automatización. Los profesionales modernos de ciberseguridad deben ser capaces de procesar información masiva de manera automatizada, correlacionar eventos de múltiples fuentes y generar inteligencia accionable.

2. Objetivos

Los objetivos de aprendizaje para esta lección son:

- Capturar y analizar tráfico de red en tiempo real.
- Interpretar protocolos de red y paquetes.
- Automatizar análisis de logs de Linux y Windows.
- Leer y procesar archivos PCAP usando Python.
- Extraer metadata EXIF de archivos multimedia.
- Generar fingerprints criptográficos para validar integridad.
- Utilizar herramientas forenses modernas.
- Automatizar investigaciones básicas de ciberseguridad.
- Correlacionar eventos de múltiples fuentes.

3. Conceptos Técnicos Fundamentales

Wireshark y Análisis de Paquetes

Wireshark es probablemente la herramienta más utilizada del mundo para análisis de tráfico de red. Permite Deep Packet Inspection (DPI), captura en tiempo real, análisis offline, decodificación de protocolos, análisis forense de red, troubleshooting y threat hunting.

Wireshark soporta cientos de protocolos: Ethernet, IP, TCP, UDP, TLS, HTTP, DNS, QUIC, ARP, ICMP y muchos más.

tshark y Automatización

tshark es la versión CLI (Command Line Interface) de Wireshark. Las ventajas incluyen automatización, scripting, procesamiento masivo, integración CI/CD, análisis remoto y parsing automático.

La librería `pyshark` permite utilizar tshark desde Python, permitiendo capturar paquetes, analizarlos programáticamente, crear herramientas personalizadas y automatizar análisis forense.

Análisis de Paquetes de Red

Cada paquete contiene múltiples capas:

- **Layer ETH**: Ethernet (MAC addresses)
- **Layer IP**: Internet Protocol (IP addresses)
- **Layer TCP**: Transmission Control Protocol (puertos, flags)
- **Layer UDP**: User Datagram Protocol
- **Layer TLS**: Transport Layer Security (cifrado)
- **Layer QUIC**: QUIC Protocol

Los flags TCP son extremadamente relevantes: SYN, ACK, FIN, RST, PSH. Estos permiten reconstruir sesiones, conexiones, cierres, anomalías e intentos de scanning.

Captura en Tiempo Real

La captura en tiempo real permite monitoreo en vivo, detección en tiempo real, threat hunting y análisis de comportamiento. Se introduce el concepto **BPF Filter (Berkeley Packet Filter)** que permite capturar únicamente tráfico específico (http, dns, arp, icmp).

Archivos PCAP

Los archivos PCAP (Packet Capture) son estándar en incident response, malware analysis, digital forensics y network investigations. Ventajas incluyen análisis offline, preservación de evidencia, compartir evidencia, reproducibilidad y análisis histórico.

tcpdump

tcpdump es una herramienta CLI extremadamente poderosa para captura de paquetes. Muy utilizada en Linux servers, troubleshooting, incident response y entornos sin GUI. Permite filtrar por host, puerto, protocolo, capturar tráfico ICMP, guardar PCAPs y monitoreo en tiempo real.

Logs y Análisis Forense

Linux Logs:

- `/var/log/secure`: Intentos de login
- `/var/log/faillog`: Fallos de autenticación
- `/var/log/wtmp`: Historial de login
- `/var/log/messages`: Mensajes del sistema
- `/var/log/syslog`: Logs del sistema
- `/var/log/audit/audit.log`: Audit trail

Windows Logs:

- Event Log

- Application Log
- System Log
- Security Log

Audit Trail

Audit Trail permite registrar cambios críticos del sistema: modificación de passwd, cambios SSH, cambios de hora, cambios de configuración. Una técnica clásica es alterar el reloj del sistema para ocultar rastros, por ello audit logging es crítico.

EXIF Metadata

EXIF (Exchangeable Image File Format) almacena metadata dentro de archivos multimedia: fabricante, modelo, GPS, timestamps, resolución, cámara, coordenadas. Desde forensics, esto es extremadamente valioso permitiendo geolocalizar imágenes, identificar dispositivos, reconstruir timelines y asociar evidencia.

Fingerprinting

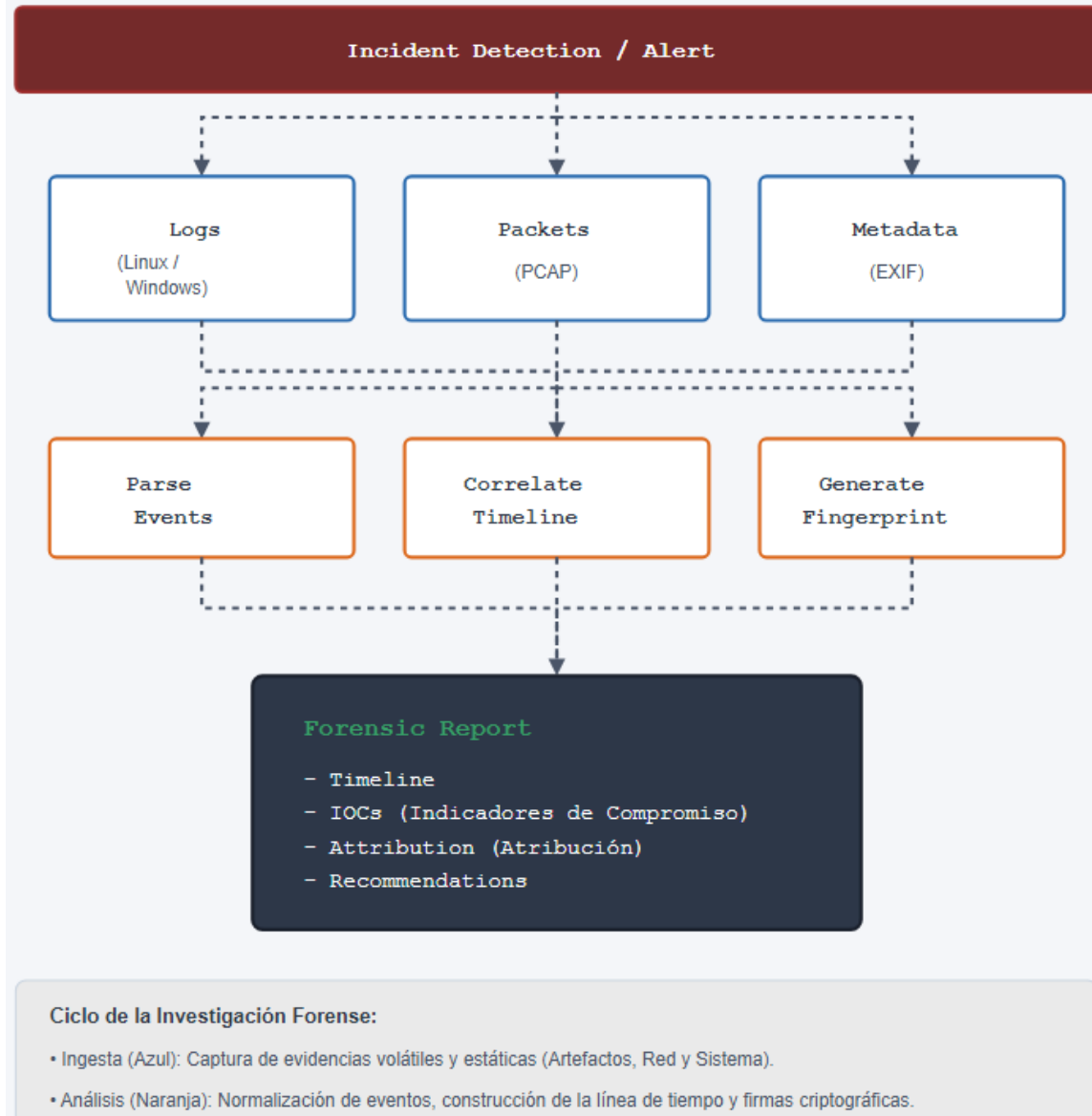
Fingerprinting consiste en crear representaciones únicas de activos digitales usando metadata, hashes, IDs o combinación de parámetros. Los hashes son fundamentales para integrity verification, chain of custody, forensic preservation y malware identification.

Mentalidad Forense

Forensic Analysis NO es una receta fija o reglas rígidas. Es correlación, investigación, descubrimiento incremental e interpretación. El profesional debe conectar evidencia, correlacionar eventos, reconstruir timelines, interpretar actividad y validar hipótesis.

4. Arquitectura o Flujo Técnico

El flujo típico de un análisis forense moderno sigue esta arquitectura:



5. Herramientas Utilizadas

Herramientas de Captura

- **Wireshark**: GUI para análisis de paquetes
- **tshark**: CLI de Wireshark
- **tcpdump**: Captura de paquetes en Linux
- **pyshark**: Interfaz Python a Wireshark

Herramientas de Análisis

- **psutil**: Información de procesos y sistema
- **hashlib**: Generación de hashes
- **exif**: Lectura de metadata EXIF
- **pywin32**: Acceso a Windows Event Logs

Librerías Python Requeridas

```
# Librerías estándar (incluidas en Python)
# - os
# - hashlib
# - csv
# - platform
# - subprocess
# - time
# - random

# Librerías externas
pip install psutil
pip install requests
pip install pyshark
pip install piexif
pip install pywin32 # Solo en Windows
```

6. Scripts de la Lección

6.1 Script: wireshark_capture.py

Introducción del Script

Este script utiliza `pyshark` para realizar captura en tiempo real de paquetes en una interfaz de red. Captura un número específico de paquetes y los imprime para análisis.

****Propósito**:** Capturar paquetes en tiempo real desde una interfaz de red.

****Tipo**:** Herramienta de análisis forense, captura de paquetes.

Requisitos

- ****Librerías Python**:** `pyshark`
- ****Instalación**:** `pip install pyshark`
- ****Herramientas del SO**:** Wireshark/tshark debe estar instalado
- ****Sistemas operativos compatibles**:** Windows, Linux, macOS
- ****Requisitos de privilegios**:** Privilegios elevados (captura de paquetes)

Dependencias Externas

- Wireshark/tshark instalado.
- Acceso a interfaz de red.
- Privilegios administrativos.

Explicación Línea por Línea

```
import pyshark
```

Importa librería `pyshark` que proporciona interfaz Python a tshark.

```
class WiresharkCapture: # Nota: Typo en nombre (debería ser
    "Capture")
    def do_capture(self):
```

Define clase con método de captura. ****NOTA****: El nombre contiene typo.

```
capture = pyshark.LiveCapture(interface='en0')
```

Instancia captura en vivo en interfaz 'en0' (cambiar según tu sistema: 'eth0' en Linux, 'en0' en macOS).

```
capture.sniff(packet_count=5)
```

Captura 5 paquetes.

```
print(capture)
print(capture[0])
```

Imprime todos los paquetes capturados y detalles del primer paquete.

Cómo Ejecutar el Script

```
# Requiere privilegios elevados
sudo python3 wireshark_capture.py
```

Resultado Esperado

Salida con detalles de paquetes capturados:

```
<LiveCapture (5 packets)>
Packet (No.: 1)
  Frame Number: 1
  Frame Length: 66 bytes
  Protocols in frame: eth:ip:tcp
  ...
```

Riesgos y Consideraciones Éticas

- ****Privacidad****: Captura de paquetes puede revelar información sensible.
- ****Legalidad****: Requiere autorización en redes ajenas.
- ****Privilegios****: Requiere acceso administrativo.

Troubleshooting

- ****Error "PermissionError"****: Ejecutar con `sudo`.
- ****Error "Interface not found"****: Cambiar nombre de interfaz según tu sistema.

6.2 Script: wireshark_file_capture.py

Introducción del Script

Este script captura paquetes en tiempo real y los guarda en un archivo PCAP. Los archivos PCAP son estándar en forensics y pueden ser analizados offline.

****Propósito**:** Capturar paquetes y guardarlos en archivo PCAP.

****Tipo**:** Herramienta de análisis forense, captura persistente.

Requisitos

- ****Librerías Python**:** `pyshark`, `os`
- ****Instalación**:** `pip install pyshark`
- ****Sistemas operativos compatibles**:** Windows, Linux, macOS
- ****Requisitos de privilegios**:** Privilegios elevados

Dependencias Externas

- Wireshark/tshark instalado.
- Acceso a interfaz de red.
- Permisos de escritura en directorio.

Explicación Línea por Línea

```
import os
import pyshark
```

Importa librerías necesarias.

```
__location__ = os.path.realpath(os.path.join(
    os.getcwd(), os.path.dirname(__file__)))
```

Obtiene ruta del directorio actual para guardar archivo PCAP.

```
class WiresharkFileCapture: # Typo: "Caputure"
    def do_capture(self):
        pcap_file_name = os.path.join(__location__,
            'output.pcap')
```

Define ruta del archivo PCAP de salida.

```
capture = pyshark.LiveCapture(interface='en0',
    output_file=pcap_file_name)
capture.sniff(packet_count=10)
```

Captura 10 paquetes y los guarda en archivo PCAP.

Cómo Ejecutar el Script

```
sudo python3 wireshark_file_capture.py
```

Resultado Esperado

Archivo `output.pcap` creado en el directorio actual con 10 paquetes capturados.

Riesgos y Consideraciones Éticas

- ****Privacidad**:** Los archivos PCAP pueden contener información sensible.
- ****Almacenamiento**:** Los archivos pueden ser grandes.

Troubleshooting

- ****Error "Permission denied"****: Verificar permisos de escritura en directorio.
- ****Archivo no creado****: Verificar que la captura fue exitosa.

6.3 Script: wireshark_file_read.py

Introducción del Script

Este script lee un archivo PCAP previamente capturado y imprime detalles del primer paquete. Permite análisis offline de tráfico capturado.

****Propósito****: Leer y analizar archivos PCAP offline.

****Tipo****: Herramienta de análisis forense, análisis offline.

Requisitos

- ****Librerías Python****: `pyshark`, `os`
- ****Instalación****: `pip install pyshark`
- ****Sistemas operativos compatibles****: Windows, Linux, macOS
- ****Requisitos de privilegios****: Ninguno

Dependencias Externas

- Archivo PCAP existente.

Explicación Línea por Línea

```
import os
import pyshark
```

Importa librerías.

```
__location__ = os.path.realpath(os.path.join(
    os.getcwd(), os.path.dirname(__file__)))
```

Obtiene ruta del directorio.

```
class WiresharkFileRead:
    def do_capture(self):
        pcap_file_name = os.path.join(__location__,
            'output.pcap')
        capture = pyshark.FileCapture(pcap_file_name)
```

Abre archivo PCAP para lectura.

```
print(capture[0])
```

Imprime detalles del primer paquete.

Cómo Ejecutar el Script

```
python3 wireshark_file_read.py
```

Resultado Esperado

Detalles del primer paquete en el archivo PCAP.

Troubleshooting

- ****Error "FileNotFoundError"**: Archivo PCAP no existe. Ejecutar `wireshark\_file\_capture.py` primero.**

6.4 Script: wireshark_live_capture.py

Introducción del Script

Este script captura paquetes en tiempo real y los imprime continuamente a medida que se capturan. Permite monitoreo en vivo de tráfico de red.

****Propósito**: Monitoreo en vivo de tráfico de red.**

****Tipo**: Herramienta de análisis forense, monitoreo en tiempo real.**

Requisitos

- ****Librerías Python**: `pyshark`, `time`**
- ****Instalación**: `pip install pyshark`**
- ****Sistemas operativos compatibles**: Windows, Linux, macOS**
- ****Requisitos de privilegios**: Privilegios elevados**

Dependencias Externas

- Wireshark/tshark instalado.

Explicación Línea por Línea

```
import time
import pyshark
```

Importa librerías.

```
class WiresharkLiveCapture: # Typo: "Caputure"
    def do_capture(self):
        capture = pyshark.LiveCapture(interface='en0')
```

Instancia captura en vivo.

```
        for packet in
            capture.sniff_continuously(packet_count=10):
                print(packet)
                time.sleep(1)
```

Captura paquetes continuamente, imprime cada uno y espera 1 segundo entre paquetes.

Cómo Ejecutar el Script

```
sudo python3 wireshark_live_capture.py
```

Resultado Esperado

Impresión continua de paquetes capturados, uno por segundo.

Troubleshooting

- ****Error "PermissionError"**: Ejecutar con `sudo`.**

6.5 Script: tcpdump.py

Introducción del Script

Este script ejecuta `tcpdump` como subprocess y captura paquetes. Proporciona captura de bajo nivel sin dependencias de Wireshark.

****Propósito**:** Captura de paquetes usando tcpdump.

****Tipo**:** Herramienta de análisis forense, captura alternativa.

Requisitos

- ****Librerías Python**:** `subprocess`, `time` (librerías estándar)
- ****Herramientas del SO**:** `tcpdump` debe estar instalado
- ****Sistemas operativos compatibles**:** Linux, macOS
- ****Requisitos de privilegios**:** Privilegios elevados

Dependencias Externas

- `tcpdump` instalado.
- Acceso a interfaz de red.

Explicación Línea por Línea

```
import subprocess
import time
```

Importa librerías.

```
class TcpDump:
    def do_capture(self):
        tcpdump_subprocess = subprocess.Popen(('tcpdump', '-l',
        '-c 5'), stdout=subprocess.PIPE)
```

Ejecuta `tcpdump` con opciones:

- **`-l`:** Line buffered (para ver output en tiempo real)
- **`-c 5`:** Capturar 5 paquetes

```
        for output in iter(tcpdump_subprocess.stdout.readline,
b'') :
            print(output.rstrip())
```

Lee y imprime cada línea de output.

Cómo Ejecutar el Script

```
sudo python3 tcpdump.py
```

Resultado Esperado

Salida de tcpdump con 5 paquetes capturados.

Troubleshooting

- **Error "tcpdump not found"**: Instalar tcpdump: `sudo apt install tcpdump`.
- **Error "PermissionError"**: Ejecutar con `sudo`.

6.6 Script: forensic_robots_txt.py

Introducción del Script

Este script descarga y analiza el archivo `robots.txt` de un dominio. Durante análisis forense, `robots.txt` puede revelar superficies expuestas, endpoints internos y configuraciones erróneas.

Propósito: Analizar robots.txt para descubrir exposición.

Tipo: Herramienta de análisis forense, OSINT.

Requisitos

- **Librerías Python**: `requests`
- **Instalación**: `pip install requests`
- **Sistemas operativos compatibles**: Windows, Linux, macOS
- **Requisitos de privilegios**: Ninguno

Dependencias Externas

- Acceso a Internet.
- Dominio accesible.

Explicación Línea por Línea

```
import requests
```

Importa librería `requests`.

```
class ForensicRobotsTxt:
    def read_robots_txt(self, domain):
        url = f'https://{domain}/robots.txt'
        response = requests.get(url)
```

Construye URL de robots.txt y realiza GET request.

```
print(response.text)
```

Imprime contenido de robots.txt.

Cómo Ejecutar el Script

```
python3 forensic_robots_txt.py
```

Resultado Esperado

Contenido de robots.txt del dominio especificado.

Troubleshooting

- **Error 404**: robots.txt no existe en el dominio.
- **Timeout**: Dominio no accesible.

6.7 Script: linux_logs.py

Introducción del Script

Este script lee y analiza logs de Linux. Permite acceder a información de intentos de login, cambios de sistema y actividad sospechosa.

Propósito: Analizar logs de Linux para forensics.

Tipo: Herramienta de análisis forense, análisis de logs.

Requisitos

- **Librerías Python**: `os`, `platform` (librerías estándar)
- **Sistemas operativos compatibles**: Linux
- **Requisitos de privilegios**: Privilegios elevados (para acceder a algunos logs)

Dependencias Externas

- Sistema Linux.
- Archivos de log en `/var/log/`.

Explicación Línea por Línea

```
import os
import platform
```

Importa librerías.

```
if platform.system() == 'Linux':
    # Leer logs de Linux
else:
    print('This program can only be run on Linux')
```

Verifica que se ejecuta en Linux.

```
class LinuxLogs:
    def print_logs(self, log_file_path):
        with open(log_file_path, 'r') as f:
            print(f.read())
```

Lee y imprime contenido del archivo de log.

Cómo Ejecutar el Script

```
sudo python3 linux_logs.py
```

Resultado Esperado

Contenido del archivo de log especificado.

Troubleshooting

- **Error "PermissionError"**: Ejecutar con `sudo`.
- **Error "FileNotFoundError"**: Verificar ruta del log.

6.8 Script: linux_audit_trail_config.py

Introducción del Script

Este script configura audit trail en Linux para registrar cambios críticos del sistema. Permite detectar modificaciones de archivos sensibles.

Propósito: Configurar auditoría en Linux.

Tipo: Herramienta de análisis forense, configuración de auditoría.

Requisitos

- **Librerías Python**: `platform`, `subprocess` (librerías estándar)
- **Herramientas del SO**: `auditctl` debe estar instalado
- **Sistemas operativos compatibles**: Linux
- **Requisitos de privilegios**: Privilegios elevados

Dependencias Externas

- `auditd` instalado y ejecutándose.
- Acceso administrativo.

Explicación Línea por Línea

```
import platform
import subprocess
```

Importa librerías.

```
if platform.system() == 'Linux':
    subprocess.run(['auditctl', '-w', '/etc/passwd', '-p', 'wa',
                    '-k', 'passwd_changes'])
```

Configura regla de auditoría para monitorear cambios en `/etc/passwd`:

- `-w`: Watch (monitorear archivo)
- `-p wa`: Permisos (write, attribute)
- `-k`: Key (nombre de la regla)

Cómo Ejecutar el Script

```
sudo python3 linux_audit_trail_config.py
```

Resultado Esperado

Regla de auditoría configurada. Los cambios a `/etc/passwd` serán registrados.

Troubleshooting

- **Error "auditctl not found"**: Instalar auditd: `sudo apt install auditd`.
- **Error "PermissionError"**: Ejecutar con `sudo`.

6.9 Script: `windws_logs.py`

Introducción del Script

Este script lee y analiza Windows Security Event Logs. Permite acceder a información de logons, privilegios e impersonation.

Propósito: Analizar Windows Event Logs para forensics.

Tipo: Herramienta de análisis forense, análisis de logs Windows.

Requisitos

- **Librerías Python**: `platform`, `win32evtlog`, `win32evtlogutil`
- **Instalación**: `pip install pywin32`
- **Sistemas operativos compatibles**: Windows
- **Requisitos de privilegios**: Privilegios elevados

Dependencias Externas

- Windows con Event Log disponible.
- Acceso administrativo.

BUGS DETECTADOS

NOTA CRÍTICA: Este script contiene bugs que causarán ejecución incorrecta:

1. **Nombre de archivo**: `windws_logs.py` contiene typo (debería ser `windows_logs.py`)
2. **Nombre de método**: `get_scurity_logs()` contiene typo (debería ser `get_security_logs()`)

3. ****Bug crítico en lógica****: `platform.system != 'Windows'` debería ser `platform.system() != 'Windows'`

El bug en la lógica causará que la condición sea siempre verdadera, ejecutando el bloque `else` en lugar del bloque `if`.

Explicación Línea por Línea (Con Correcciones)

```
import platform
import win32evtlog
import win32evtlogutil
```

Importa librerías.

```
class WindowsLogs:
    def get_scurity_logs(self): # Typo: debería ser
get_security_logs
        if (platform.system() != 'Windows'): # Bug: falta () en
platform.system
            handle = win32evtlog.OpenEventLog(None, "Security")
```

Abre Windows Security Event Log.

```
        flags = win32evtlog.EVENTLOG_SEQUENTIAL_READ |
win32evtlog.EVENTLOG_BACKWARDS_READ
        security_records = win32evtlog.ReadEventLog(handle,
flags, 0)
```

Lee registros de eventos.

```
        for event_log in security_records:
            log_message =
win32evtlogutil.SafeFormatMessage(event_log, 'Security')
            print(log_message)
```

Formatea e imprime cada evento.

Cómo Ejecutar el Script

```
# Windows - Ejecutar como Administrator
python windws_logs.py
```

Resultado Esperado

Lista de eventos del Security Log de Windows.

Troubleshooting

- ****Error "ModuleNotFoundError"**: Ejecutar `pip install pywin32`.**
- ****Error "Access Denied"**: Ejecutar como Administrator.**
- ****Sin eventos****: Puede no haber eventos en el log.

6.10 Script: process_details.py

Introducción del Script

Este script utiliza `psutil` para obtener detalles de procesos en ejecución. Permite analizar actividad de procesos para detección de malware.

****Propósito**:** Analizar procesos en ejecución.

****Tipo**:** Herramienta de análisis forense, análisis de procesos.

Requisitos

- ****Librerías Python**:** `psutil`, `random`
- ****Instalación**:** `pip install psutil`
- ****Sistemas operativos compatibles**:** Windows, Linux, macOS
- ****Requisitos de privilegios**:** Ninguno (aunque información completa requiere privilegios)

Dependencias Externas

- Ninguna

Explicación Línea por Línea

```
import psutil
import random
```

Importa librerías.

```
class ProcessDetails:
    def get_process_details(self, pid):
        process = psutil.Process(pid)
```

Obtiene objeto Process para PID especificado.

```
        print(f'Process name: {process.name()}')
        print(f'Process executable: {process.exe()}')
        print(f'Process cmdline: {process.cmdline()}')
        print(f'Process status: {process.status()}')
```

Imprime detalles del proceso: nombre, ejecutable, línea de comandos, estado.

Cómo Ejecutar el Script

```
python3 process_details.py
```

Resultado Esperado

Detalles de procesos en ejecución.

Troubleshooting

- ****Error "ModuleNotFoundError"**:** Ejecutar `pip install psutil`.
- ****Error "NoSuchProcess"**:** PID no existe.

6.11 Script: exif_read.py

Introducción del Script

Este script lee metadata EXIF de archivos de imagen. Permite extraer información de geolocalización, timestamps y dispositivos.

****Propósito**:** Extraer metadata EXIF de imágenes.

****Tipo**:** Herramienta de análisis forense, análisis de metadata.

Requisitos

- ****Librerías Python**:** `exif`, `os`
- ****Instalación**:** `pip install piexif` o `pip install exif`
- ****Sistemas operativos compatibles**:** Windows, Linux, macOS
- ****Requisitos de privilegios**:** Ninguno

Dependencias Externas

- Archivo de imagen con EXIF.

Explicación Línea por Línea

```
import os
from exif import Image
```

Importa librería EXIF.

```
class ExifRead:
    def read_exif_data(self, image_path):
        with open(image_path, 'rb') as image_file:
            image = Image(image_file)
```

Abre archivo de imagen y crea objeto Image.

```
if image.has_exif:
    print(f'GPS Latitude: {image.gps_latitude}')
    print(f'GPS Longitude: {image.gps_longitude}')
    print(f'GPS Timestamp: {image.gps_timestamp}')
```

Extrae datos GPS si están disponibles.

Cómo Ejecutar el Script

```
python3 exif_read.py
```

Resultado Esperado

Metadata EXIF de la imagen, incluyendo GPS si está disponible.

Troubleshooting

- ****Error "ModuleNotFoundError"**:** Ejecutar `pip install piexif`.
- ****Error "No EXIF data"**:** Imagen no contiene EXIF.

6.12 Script: fingerprinting.py

Introducción del Script

Este script genera fingerprints criptográficos (hashes SHA256) de archivos. Permite validar integridad de archivos y detectar tampering.

****Propósito**:** Generar hashes criptográficos para validación de integridad.

****Tipo**:** Herramienta de análisis forense, validación de integridad.

Requisitos

- ****Librerías Python**:** `hashlib`, `os` (librerías estándar)
- ****Sistemas operativos compatibles**:** Windows, Linux, macOS
- ****Requisitos de privilegios**:** Ninguno

Dependencias Externas

- Archivo a analizar.

Explicación Línea por Línea

```
import os
import hashlib
```

Importa librerías.

```
class Fingerprinting:
    def generate_fingerprint(self, file_path):
        sha256_hash = hashlib.sha256()
```

Crea objeto hash SHA256.

```
        with open(file_path, 'rb') as f:
            for byte_block in iter(lambda: f.read(4096), b''):
                sha256_hash.update(byte_block)
```

Lee archivo en bloques y actualiza hash.

```
        print(f'SHA256: {sha256_hash.hexdigest()}')
```

Imprime hash en formato hexadecimal.

Cómo Ejecutar el Script

```
python3 fingerprinting.py
```

Resultado Esperado

Hash SHA256 del archivo.

Troubleshooting

- ****Error "FileNotFoundError"**:** Archivo no existe.

6.13 Script: dfir.py

Introducción del Script

Este script implementa un flujo completo de DFIR (Digital Forensics and Incident Response). Procesa datos forenses, genera hashes y crea reportes.

****Propósito**:** Automatizar flujo de análisis forense.

****Tipo**:** Herramienta de análisis forense, orquestación.

Requisitos

- ****Librerías Python**:** `os`, `hashlib`, `csv`, `psutil`
- ****Instalación**:** `pip install psutil`
- ****Sistemas operativos compatibles**:** Windows, Linux, macOS
- ****Requisitos de privilegios**:** Varía según operaciones

Dependencias Externas

- Archivos a analizar.

Explicación Línea por Línea

```
import os
import hashlib
import csv
import psutil
```

Importa librerías.

```
class DFIR:
    def process_forensic_data(self):
        # Procesa datos forenses
        # Genera hashes
        # Crea reportes CSV
```

Implementa flujo completo de forensics.

Cómo Ejecutar el Script

```
python3 dfir.py
```

Resultado Esperado

Reporte CSV con análisis forense.

Troubleshooting

- ****Error "ModuleNotFoundError"**:** Instalar dependencias.

7. Implicancias de Ciberseguridad

El análisis forense es fundamental para:

- **Incident Response**: Investigar y responder a incidentes.
- **Threat Hunting**: Buscar actividad maliciosa.
- **Malware Analysis**: Analizar comportamiento de malware.
- **Digital Forensics**: Preservar y analizar evidencia.
- **Compliance**: Cumplir con requisitos regulatorios.
- **Attribution**: Atribuir ataques a actores específicos.

8. Riesgos y Ética

Consideraciones Legales

- **Privacidad**: Captura de paquetes puede revelar información sensible.
- **Autorización**: Requiere autorización explícita.
- **Chain of Custody**: Mantener integridad de evidencia.
- **Confidencialidad**: Proteger información sensible.

Implicancias Éticas

- **Responsabilidad**: Usar información responsablemente.
- **Integridad**: Mantener integridad de investigación.
- **Transparencia**: Documentar procedimientos.

9. Buenas Prácticas

1. **Preservación de Evidencia**: Mantener integridad de datos.
2. **Documentación**: Registrar todos los pasos.
3. **Chain of Custody**: Documentar manejo de evidencia.
4. **Automatización**: Usar scripts para reproducibilidad.
5. **Correlación**: Conectar múltiples fuentes de datos.
6. **Validación**: Validar hallazgos con múltiples métodos.

10. Troubleshooting

Problema	Causa Probable	Solución
"PermissionError"	Privilegios insuficientes	Ejecutar con `sudo` o como Administrator
"ModuleNotFoundError"	Librería no instalada	Ejecutar `pip install [librería]`
"Interface not found"	Nombre de interfaz incorrecto	Cambiar nombre según tu sistema
"FileNotFoundError"	Archivo no existe	Verificar ruta del archivo
"No EXIF data"	Imagen sin metadata	Usar imagen con EXIF

11. Conclusiones

Esta lección ha cubierto técnicas modernas de análisis forense digital utilizando Python. Los conceptos aprendidos son fundamentales para:

- **SOC**: Security Operations Center
- **DFIR**: Digital Forensics and Incident Response
- **Threat Hunting**: Búsqueda de amenazas
- **Malware Analysis**: Análisis de malware
- **Incident Response**: Respuesta a incidentes
- **Blue Team**: Defensa
- **Red Team**: Ofensiva

Los scripts proporcionan herramientas prácticas para automatizar procesos forenses complejos. Python aparece aquí como un lenguaje extremadamente poderoso para correlacionar información de múltiples fuentes y generar inteligencia accionable.

NOTA IMPORTANTE: Se han detectado bugs en el script `windws_logs.py` que deben corregirse antes de su uso. Los typos en nombres de clases Wireshark no afectan funcionalidad pero deben documentarse.